A Semantic Comparison and Evaluation System for Knowledge Graph and Artificial Intelligence-Based Informatics Education Curricula

Ali Ekrem Yapıcı (05220001018), Ceyda Yapar (05210000286), Berat Altun (05210000957)

Department of Computer Engineering, Ege University, 35100 Bornova, İzmir

Student e-mail:

05210000957@ogrenci.ege.edu.tr
05210000286@ogrenci.ege.edu.tr
05220001018@ogrenci.ege.edu.tr

Advisor: Prof. Dr. Murat Osman ÜNALIR

Advisor e-mail: murat.osman.unalir@ege.edu.tr

Submission Date: February 27, 2026

Abstract

This study presents the development of an AI-supported system for modeling, comparing, and evaluating informatics-oriented undergraduate curricula in Türkiye using knowledge graph and natural language processing (NLP) technologies. Within the scope of the project, the computer engineering curricula of five universities (Ege University, Middle East Technical University, Özyeğin University, Selçuk University, and TOBB University of Economics and Technology) were structured in JSON format and ingested into a Neo4j knowledge graph. Semantic vector representations were generated for course contents, learning outcomes, and program outcomes using Sentence Transformers. A total of 11 comparison metrics were developed and presented to users through a FastAPI-based web application featuring a premium dark-themed UI. Additionally, an AI-powered chatbot assistant was integrated using the Groq API (Llama 3.3 70B) to enable natural language querying of the knowledge graph.

Keywords: Knowledge Graph, Natural Language Processing, Curriculum Comparison, Neo4j, Semantic Similarity, Artificial Intelligence

1. Introduction

The rapid growth of information technologies has led to the establishment of numerous informatics-related undergraduate programs across Turkish universities. While these programs share similar foundational goals, their curricula often differ significantly in terms of course content, credit distribution, prerequisite structures,

and learning outcomes. Evaluating and comparing these curricula manually is a time-intensive, error-prone, and non-standardized process.

This graduation thesis project aims to develop an AI-supported semantic comparison and evaluation system for informatics education curricula. The system leverages knowledge graphs to model the structural and semantic relationships within university curricula and employs NLP techniques to calculate similarity scores at multiple levels (courses, learning outcomes, program outcomes, weekly topics). The key objectives of this work are:

1. Data Collection and Structuring: Gathering comprehensive curriculum data from multiple university departments and structuring it in a standardized JSON format covering courses, ECTS credits, theory/practice hours, prerequisites, learning outcomes, program outcomes, academic staff, and weekly topics.

2. Knowledge Graph Construction: Building an ontology-based knowledge graph in Neo4j that represents universities, faculties, departments, courses, learning outcomes, program outcomes, academic staff, course types, and categories with their inter-relationships.

3. Semantic Analysis: Generating NLP-based embeddings using the paraphrase-multilingual-MiniLM-L12-v2 Sentence Transformer model for course descriptions, learning outcomes, program outcomes, and weekly topics to enable semantic similarity computation.

4. Multi-Metric Comparison: Developing 11 distinct comparison metrics that cover structural, semantic, and statistical dimensions of curriculum evaluation.

5. Web Application and AI Assistant: Building an interactive web platform with a dashboard, comparison tools, and an AI-powered chatbot for natural language querying.

The significance of this project lies in its ability to provide a standardized, data-driven, and traceable infrastructure for curriculum evaluation. This contributes to accreditation processes (MÜDEK/YÖKAK), inter-university curriculum alignment efforts, and educational policy development at the national level.

2. Literature Review

The theoretical foundation of this project draws upon several interconnected fields:

Knowledge Graphs and Ontology Modeling: Knowledge graphs provide a powerful mechanism for representing structured, interconnected information. Neo4j, a leading graph database, enables efficient storage and querying of complex hierarchical relationships using the Cypher query language [1]. Ontology-based modeling of educational data has been explored in various studies for curriculum representation [2].

Natural Language Processing and Semantic Similarity: Sentence Transformers [3] extend the BERT architecture to generate semantically meaningful sentence embeddings, enabling computation of cosine similarity between textual data. The paraphrase-multilingual-MiniLM-L12-v2 model used in this project supports Turkish language text, making it suitable for analyzing Turkish-language course descriptions and learning outcomes. The scikit-learn library provides efficient cosine similarity computation on embedding matrices [4].

Curriculum Standards and Frameworks: The ACM/IEEE Computing Curricula 2020 (CC2020) [5] provides competency-based guidelines for computing education. The European Qualifications Framework (EQF) [6]

establishes a common reference framework for qualification levels. UNESCO's ISCED-F 2013 [7] classification provides a standard taxonomy for education fields. These frameworks inform the comparison metrics and categorization system used in this project.

Existing Curriculum Analysis Tools: While several studies have addressed curriculum mapping and comparison, most rely on manual keyword matching or simple text overlap metrics. This project advances the state of the art by combining knowledge graph modeling with deep learning-based semantic embeddings to enable multi-dimensional comparison across structural, semantic, and statistical dimensions.

3. Methodology and Technologies

3.1 System Architecture

The system follows a three-layer architecture:

1. Data Layer: JSON-formatted curriculum data → Neo4j Knowledge Graph with embedded vectors

2. Processing Layer: Python-based ingestion pipeline, Sentence Transformers for embedding generation, and a comparison engine with 11 metrics

3. Presentation Layer: FastAPI REST API backend + HTML/CSS/JavaScript frontend with AI chatbot

3.2 Technologies Used

Component	**Technology**	**Purpose**
Graph Database	Neo4j ≥ 5.0	Knowledge graph storage and Cypher queries
NLP Embeddings	Sentence Transformers (paraphrase-multilingual-MiniLM-L12-v2)	Multilingual semantic vector generation
Similarity Computation	scikit-learn (cosine_similarity)	Pairwise embedding similarity
Backend API	FastAPI ≥ 0.100, Uvicorn	REST API endpoints with Swagger documentation
AI Chatbot	Groq API (Llama 3.3 70B Versatile)	Natural language knowledge graph querying
Frontend	HTML5, CSS3, JavaScript (Vanilla)	Interactive dashboard and comparison UI
Data Format	JSON	Standardized curriculum data exchange
Environment	Python 3.12, python-dotenv	Configuration and dependency management

3.3 Knowledge Graph Schema

The Neo4j knowledge graph uses the following node types and relationships:

Node Types:

- University — name, type (state/private), language
- Faculty — faculty name, university association
- Department — department name, university association
- Course — code, name, ECTS, theory/practice hours, semester, year, language, purpose, description, weekly topics, resources, embedding, topics_embedding
- AcademicStaff — professor, associate professor, assistant professor, lecturer, research assistant counts
- ProgramOutcome — text, index, embedding
- LearningOutcome — text, index, embedding
- CourseType — mandatory (zorunlu) / elective (seçmeli)
- Category — course category classification

Relationships:

- University -[:HAS_FACULTY]-> Faculty
- Faculty -[:HAS_DEPARTMENT]-> Department
- Department -[:OFFERS]-> Course
- Department -[:HAS_STAFF]-> AcademicStaff
- Department -[:HAS_PROGRAM_OUTCOME]-> ProgramOutcome
- Course -[:HAS_OUTCOME]-> LearningOutcome
- Course -[:HAS_TYPE]-> CourseType
- Course -[:BELONGS_TO]-> Category
- Course -[:REQUIRES]-> Course (prerequisite)

4. Completed Works

4.1 Analysis

The analysis phase involved identifying and evaluating curriculum data sources across Turkish universities. Five universities were selected to represent a diverse cross-section of Turkish higher education:

University	**Type**	**Language**	**Courses**	**Total ECTS**
Ege University	State	Turkish	69	—
Middle East Technical University (METU)	State	English	116	—
Özyeğin University	Private	English	92	486
Selçuk University	State	Turkish	60	258
TOBB University of Economics and Technology	Private	Turkish	136	793

A standardized JSON schema was designed to capture all curriculum dimensions: university metadata, faculty/department hierarchy, academic staff counts, program outcomes, and per-course details (code, name, ECTS, theory/practice hours, semester/year, language, type, purpose, description, weekly topics, learning outcomes, prerequisites, resources, and categories).

4.2 Design

The system design centered on three core components:

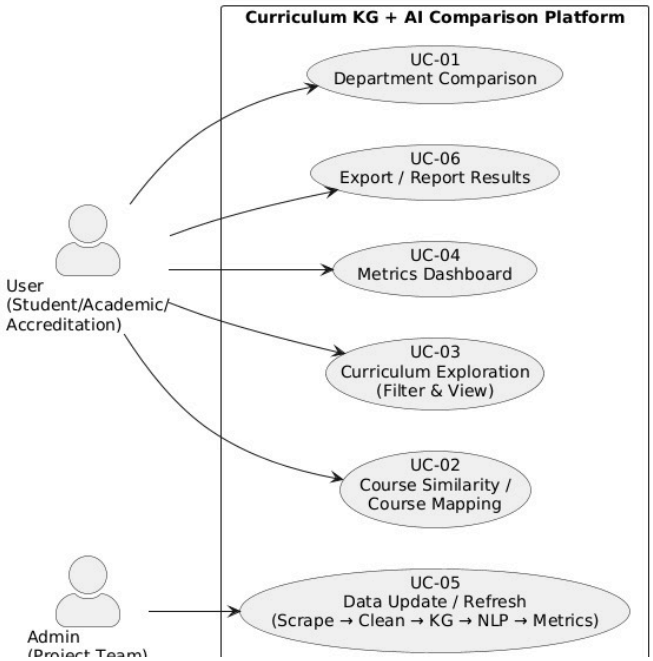
Knowledge Graph Ontology: An ontology was designed with 9 node types and 9 relationship types to capture the full structural hierarchy of university curricula. The schema supports multi-university data with proper namespacing (each node includes a university association to prevent cross-contamination).

Comparison Metric Design: Eleven comparison metrics were designed across three categories:

- Semantic Metrics (NLP-based): Course Similarity, Program Outcome Similarity, Learning Outcome Similarity, Curriculum Coverage
- Structural Metrics: Prerequisite Complexity, Semester Distribution
- Statistical Metrics: Staff Comparison, Workload Comparison, Mandatory vs. Elective Ratio, Language Distribution, Resource Overlap

Web Application Design: A single-page application with three primary sections was designed: Dashboard (aggregate statistics and charts), Comparison (11-metric analysis interface), and AI Chatbot (natural language querying).

4.2.1 Use Cases



UC-01 — Department Comparison (Fully Dressed)

Primary Actor: User

Trigger: The user opens the “Compare” screen.

Preconditions

1. At least two departments are registered in the system.

2. Metrics for these departments have been computed.

Main Success Scenario

1. The user selects Department A.

2. The user selects Department B.

3. The system loads the metrics (Similarity/Coverage/Up-to-dateness).

4. The system displays summary differences (e.g., “Coverage CS core: A %X, B %Y”).

5. The user may optionally open detailed course matches.

Postconditions (Success)

* The comparison results are displayed.

Alternative / Exception Flows

* A1: If metrics are unavailable → The system shows a “Pipeline must be executed” warning and displays the latest dataset version.

* E1: If department selection cannot be found → The system requests a new selection.

UC-02 — Course Similarity / Course Mapping (Fully Dressed)

Primary Actor: User

Goal: To find and examine the most similar courses in other departments for a selected course.

Trigger: The user selects “Find Similar Courses” on a course detail page.

Preconditions

1. Courses are loaded into the system (course metadata + description/outcomes).

2. Embeddings have been generated and indexed (cached).

3. Target department(s) or university filters are selectable.

Main Success Scenario

1. The user selects a University/Department (optional filter).

2. The user selects a Course (or arrives from course detail).

3. The system retrieves the embedding of the selected course.

4. The system filters the candidate course pool (e.g., selected departments, language, ECTS range).

5. The system computes Top-k (e.g., k=10) similar courses using cosine similarity.

6. The system lists the results (course name, code, department, similarity score).

7. The user selects a match; the system displays a side-by-side comparison of content/outcomes.

Postconditions (Success)

* The user has viewed similar courses and examined at least one match.

* The system may record view logs or usage analytics.

Alternative / Exception Flows

* A1 — Missing embedding: If the embedding of the selected course is unavailable, the system shows an "Embedding missing" warning and queues recomputation (or redirects to the admin pipeline).

* A2 — Excessively large candidate pool: If the candidate set is too large, the system applies fast filtering (e.g., Topic/Keyword) before similarity computation.

* E1 — Empty course content: If description/outcomes are missing, the system performs weak matching using metadata only and marks the result as "low confidence."

* E2 — No results: If insufficient candidates exist for Top-k, the system suggests relaxing filters (e.g., removing department filter).

Business Rules / Notes

* BR1: Similarity scores are normalized within the range 0–1.

* BR2: Duplicate courses within the same university are deduplicated.

* BR3: Results must be traceable through source URLs.

UC-03 — Curriculum Exploration (Fully Dressed)

Primary Actor: User

Goal: To explore the curriculum of a selected university/department, filter courses, and view course details.

Trigger: The user opens the "Explore Curriculum" page.

Preconditions

1. University/department and course data are loaded into the system.

2. Course metadata fields (code, name, ECTS, etc.) are available.

Main Success Scenario

1. The user selects a University.

2. The user selects a Department/Program.

3. The system retrieves the course list of the selected program.

4. The user applies filters (semester, ECTS range, language, mandatory/elective if available, keyword, topic/area).

5. The system displays the filtered course list.

6. The user selects a course.

7. The system displays course details: description, learning outcomes, ECTS, source URL, and related topic/competency tags (if available).

Postconditions (Success)

* The user has viewed the curriculum and opened at least one course detail.

Alternative / Exception Flows

* A1 — Empty filter result: The system displays “No results found” and suggests relaxing filters.

* A2 — Missing fields: If description/outcomes are missing, the system shows a “Missing content” label and provides the source page link.

* E1 — Program not found: The system returns to the selection screen and requests reselection.

Business Rules / Notes

* BR1: Each course record must be traceable with source URL and scrape timestamp.

* BR2: Duplicate courses across pages are deduplicated using code + university as the base identifier.

UC-04 — Metrics Dashboard (Fully Dressed)

Primary Actor: User

Goal: To view departmental metrics (Similarity/Coverage/Up-to-dateness) in a single interface and examine detailed breakdowns if needed.

Trigger: The user opens the “Dashboard” page or selects “View Metrics” from a department page.

Preconditions

1. Metrics for the selected department have been computed (at least one dataset version).

2. Course data and topic/competency tags are available.

Main Success Scenario

1. The user selects a University/Department.

2. The system loads metrics from the latest dataset version.

3. The system displays metric cards:

    * Coverage Score

    * Up-to-dateness Score

    * (Optional) Average Similarity to a selected benchmark department

4. The user opens Details.

5. The system shows metric breakdowns (e.g., coverage by competency area, courses contributing to up-to-dateness).

6. The user optionally selects a benchmark (e.g., comparison with METU).

7. The system displays differences and summary interpretations.

Postconditions (Success)

* The user has viewed metrics and examined at least one detail.

Alternative / Exception Flows

* A1 — Metrics unavailable: The system shows “Metrics not computed,” optionally displays course lists, and suggests admin refresh.

* A2 — Outdated dataset: The system displays a “Data may be outdated” label.

* E1 — Computation error: The system shows an error code and log ID without exposing technical details.

Business Rules / Notes

* BR1: Each metric must include a short explanation of its calculation in the UI.

* BR2: Metrics are version-dependent; optional version selection should be supported.

UC-05 — Data Update / Refresh (Admin) (Fully Dressed)

Primary Actor: Admin (Project Team)

Goal: To add a new university or update existing curriculum data and execute the full pipeline.

Trigger: The admin initiates refresh from the panel or CLI.

Preconditions

1. Admin authorization is available.

2. Scraper configuration is defined.

3. Storage and database connections are operational.

Main Success Scenario

1. The admin selects:

    * new university

    * selected universities

    * all universities

2. The system starts scraping and logs source URLs and timestamps.

3. Raw data is stored (JSON/HTML snapshots).

4. Cleaning/normalization pipeline runs.

5. KG import (Neo4j upsert) executes.

6. Embeddings are generated incrementally.

7. Metrics are recomputed.

8. A new dataset version is created and reported.

Postconditions (Success)

* The new dataset version becomes active.

* Logs and error reports are stored.

Alternative / Exception Flows

* A1 — Access blocked: University marked partial; previous version preserved.

* A2 — HTML structure changed: Marked as needing configuration update.

* E1 — Import failure: Transaction rollback; previous version retained.

* E2 — Embedding model changed: Full recomputation required and reported.

Success Metrics

* At least 15 universities successfully scraped.

* Failure rate below threshold (e.g., <5%).

* Dataset version created and queries operational.

UC-06 — Export / Report Results (Fully Dressed)

Primary Actor: User

Goal: To export comparison, metric, or similarity results (PDF/CSV/JSON) and share them.

Trigger: The user clicks the Export button.

Preconditions

1. Exportable results exist on screen.

2. Export formats are supported (minimum CSV + JSON; PDF optional).

Main Success Scenario

1. The user selects export type: CSV / JSON / PDF.

2. The user selects scope:

    * summary metrics

    * metrics + course list

    * metrics + Top-k matches

3. The system generates export content and includes dataset version.

4. Source traceability metadata is added.

5. The system provides downloadable file or link.

Postconditions (Success)

* The user obtains the exportable report with traceable dataset version.

Alternative / Exception Flows

* A1 — PDF unavailable: Suggest CSV/JSON.

* A2 — Large export: Suggest narrowing scope.

* E1 — Export failure: Show retry message.

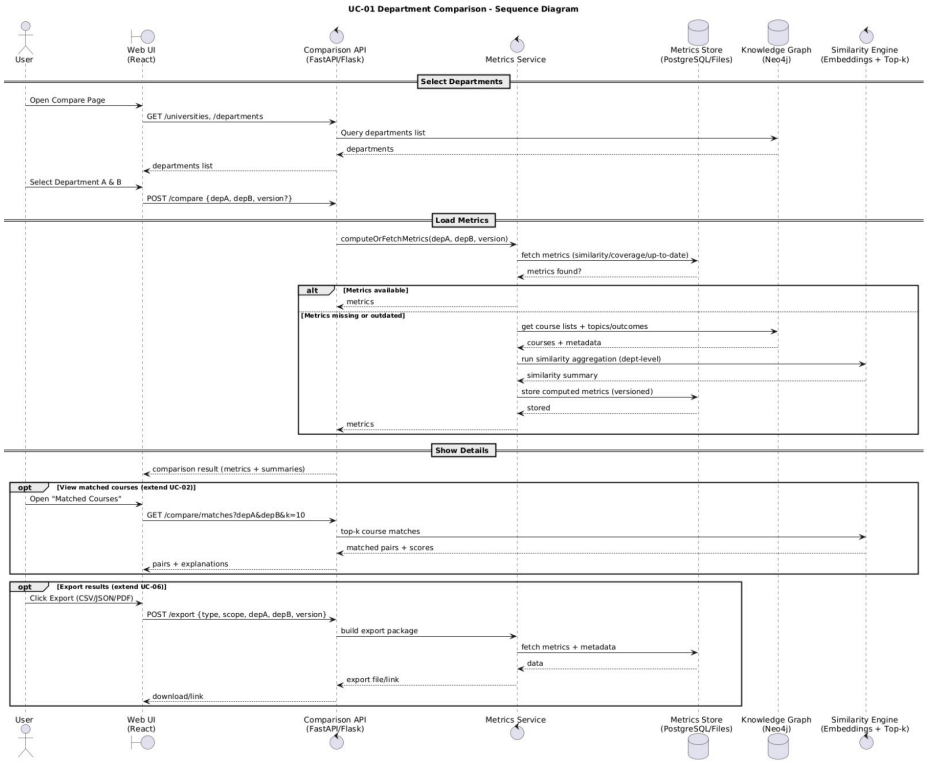
Business Rules / Notes

* BR1: Export must include metadata: dataset_version, generated_at, universities_included, metric_definitions_ref.

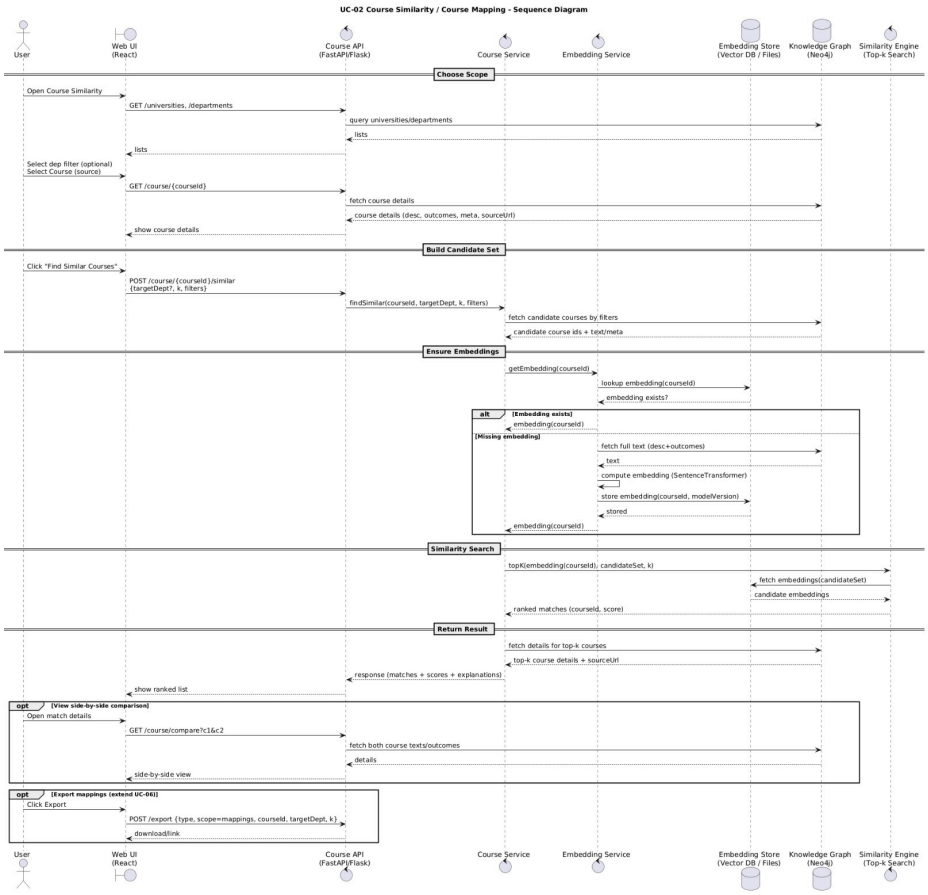
* BR2: No personal data must be included (public curriculum data only).

4.2.2 Sequence Diagrams

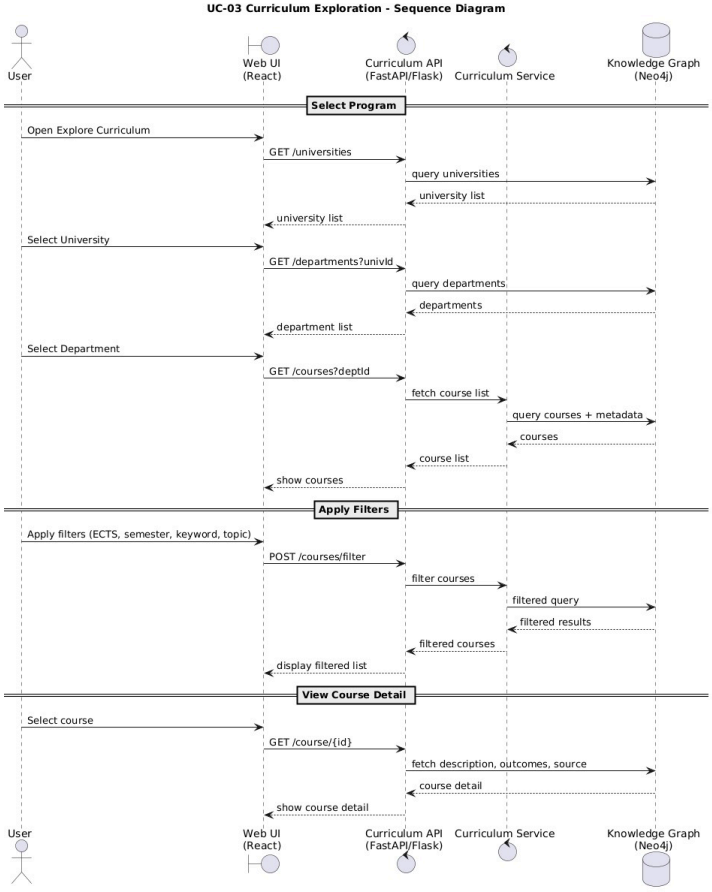
UC-01 — Department Comparison



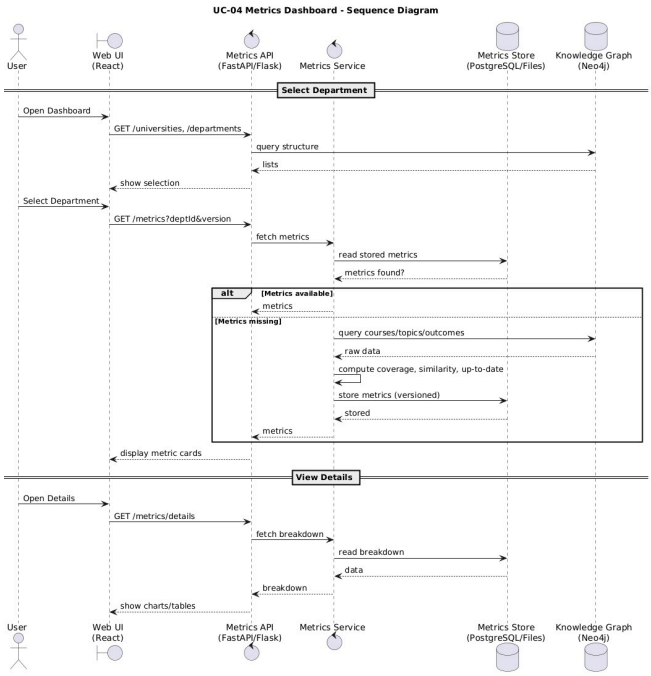
UC-02 — Course Similarity / Course Mapping



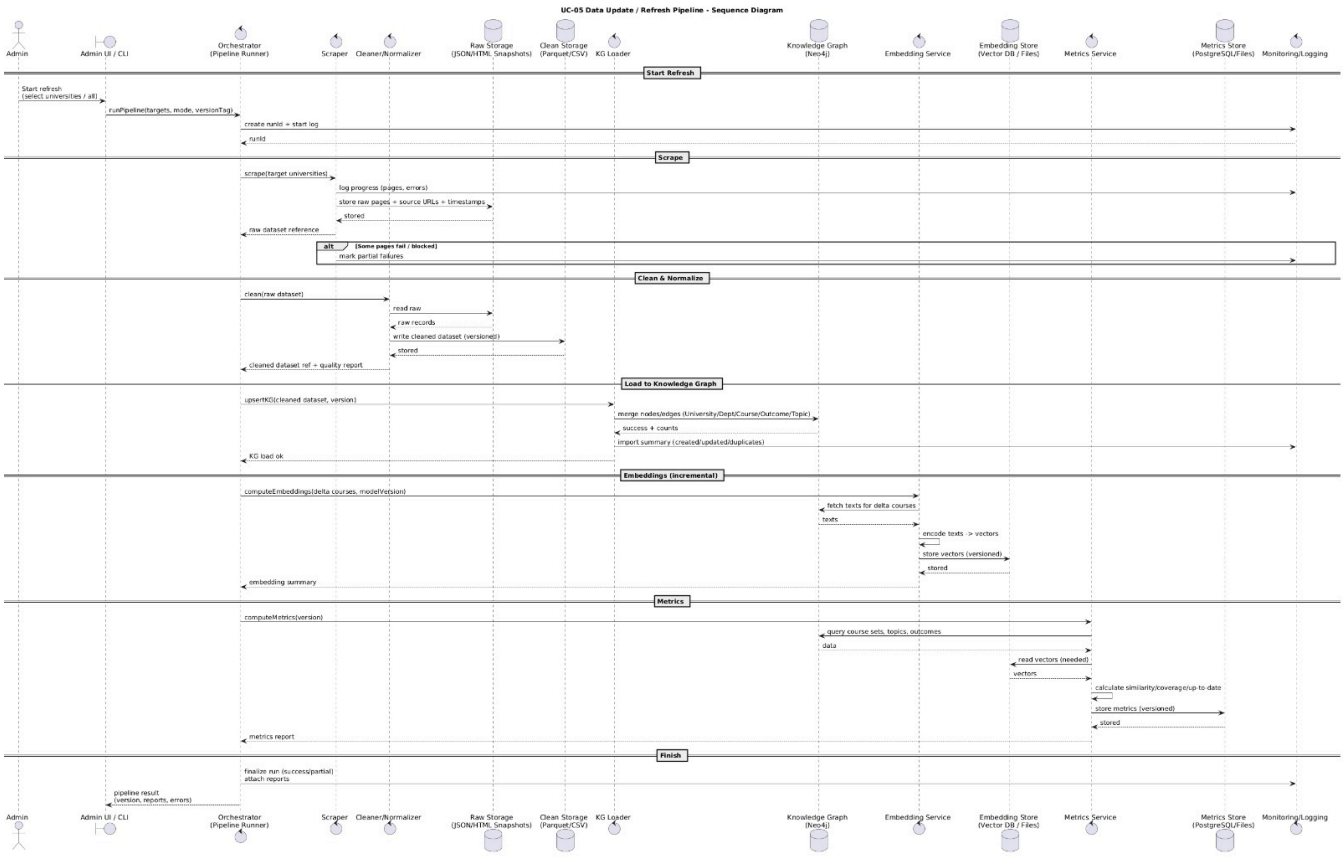
UC-03 — Curriculum Exploration



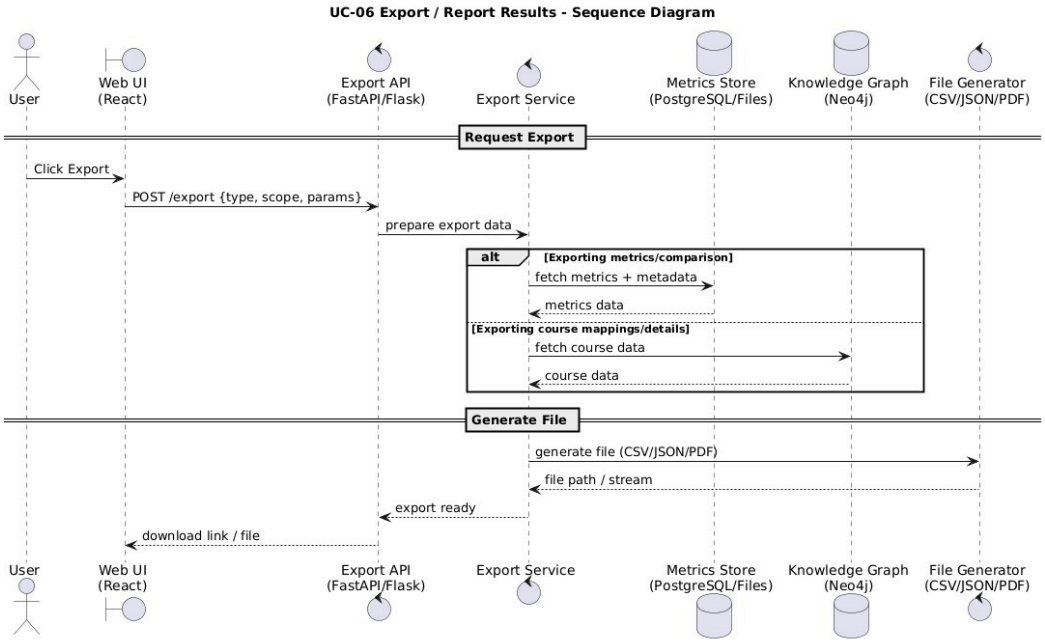
UC-04 — Metrics Dashboard



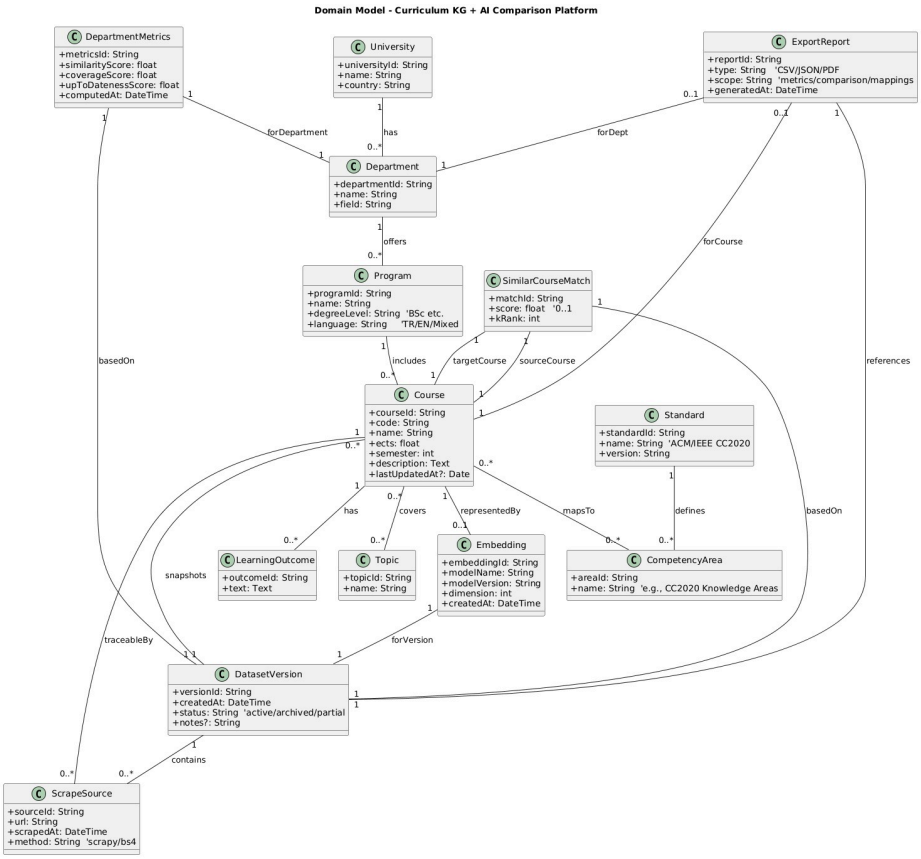
UC-05 — Data Update / Refresh (Admin)



UC-06 — Export / Report Results



4.2.3 Domain Model



4.3 Implementation

The implementation phase produced three core Python modules and a complete web frontend:

1. Data Ingestion Pipeline (`ingest.py` — 380 lines)

This module reads JSON curriculum files from the `data/` directory and ingests them into the Neo4j knowledge graph. Key capabilities include:

- Hierarchical node creation: University → Faculty → Department → Course
- Automatic embedding generation using Sentence Transformers for course descriptions, learning outcomes, program outcomes, and weekly topics
- Academic staff ingestion with title-based counts
- Prerequisites linking between course nodes
- Category and course type classification
- Transaction-based ingestion for data integrity

2. Comparison Engine (`comparison.py` — 1,116 lines)

This is the core analytical module implementing all 11 comparison metrics:

#	**Metric**	**Method**
1	Course Similarity	Cosine similarity on course description embeddings
2	Staff Comparison	Structural comparison of academic staff counts by title
3	Workload Comparison	Average ECTS, theory/practice hour ratios
4	Program Outcome Similarity	Cosine similarity on program outcome embeddings
5	Learning Outcome Similarity	Per-course learning outcome embedding comparison
6	Curriculum Coverage	Weekly topics embedding comparison; identifies unique/ shared courses
7	Prerequisite Complexity	Graph traversal depth analysis of prerequisite chains
8	Semester Distribution	ECTS and course count distribution across semesters
9	Mandatory vs. Elective Ratio	Percentage breakdown of course types
10	Language Distribution	Turkish/English course ratio analysis
11	Resource Overlap	Keyword-based textbook/resource matching

Additionally, the module includes:

- Dashboard statistics aggregation (total universities, courses, ECTS, program/learning outcomes, categories)
- University-specific chart data generation (course type distribution, semester ECTS, language breakdown, ECTS histogram)
- AI chatbot context builder that queries all knowledge graph data (faculties, departments, staff, courses, program outcomes) to provide to the LLM
- Chatbot answer generation using Groq API (Llama 3.3 70B Versatile) with knowledge graph context

3. API Backend (`main.py` — 240 lines)

A FastAPI application providing:

- 2 data endpoints (list universities, list courses)
- 2 dashboard endpoints (aggregate stats, university charts)
- 1 chatbot endpoint (POST /api/chat)
- 11 comparison endpoints (one per metric)
- CORS middleware for cross-origin access
- Static file serving for the frontend
- Swagger UI documentation at /docs

4. Web Frontend (`static/` — index.html, styles.css, app.js)

A premium, modern single-page application featuring:

- Dark glassmorphism design with CSS custom properties
- Three-section layout: Dashboard, Comparison, AI Chatbot
- Interactive charts using Chart.js (doughnut, bar, pie charts)

- Responsive design with smooth transitions and micro-animations
- Real-time API integration for all comparison metrics
- Chat interface with conversation history

4.4 Testing

Functional testing was performed at multiple levels:

- Data Integrity: Verified that all 5 university JSON files are correctly parsed and ingested into Neo4j (total: 473 courses across 5 universities)
- API Testing: All 16 API endpoints tested via Swagger UI and browser requests (200 OK responses confirmed)
- Embedding Verification: Confirmed that Sentence Transformer embeddings are generated and stored for course descriptions, learning outcomes, program outcomes, and weekly topics
- Chatbot Testing: Verified natural language querying with knowledge graph context via Groq API
- Frontend Testing: Dashboard loads correctly with aggregate statistics and charts; comparison view produces results for all 11 metrics; chatbot responds to university-related queries in Turkish

5. Planned Works

The following tasks are planned for the Spring semester (March–May 2026):

Phase	**Period**	**Tasks**
Phase 1	March 1–15	Expand dataset to 8–10 universities; validate data quality
Phase 2	March 16–31	Enhance comparison metrics with weighted scoring and normalization; add MÜDEK/YÖKAK alignment scoring
Phase 3	April 1–15	Implement advanced visualizations: interactive knowledge graph explorer, radar charts for multi-metric comparison
Phase 4	April 16–30	Add PDF report generation; implement export functionality (CSV, JSON); enhance chatbot with multi-turn conversation memory
Phase 5	May 1–15	Comprehensive system testing; performance optimization; security review
Phase 6	May 16–31	Final documentation; thesis writing; demo preparation

Key planned enhancements:

1. Data Expansion: Add curriculum data for additional universities (e.g., Hacettepe, Boğaziçi, İTÜ)

2. Advanced Analytics: Implement composite similarity scores combining multiple metrics with configurable weights

3. Accreditation Support: Map comparison results to MÜDEK criteria and generate accreditation-oriented reports

4. Graph Visualization: Interactive Neo4j graph explorer in the frontend showing curriculum relationships

5. Report Generation: Automated PDF comparison reports for stakeholders

6. User Authentication: Role-based access for different user types (academics, administrators, accreditation bodies)

References

[1] Neo4j, Inc., "Neo4j Graph Data Platform," 2024. [Online]. Available: https://neo4j.com

[2] R. Studer, V. R. Benjamins, and D. Fensel, "Knowledge Engineering: Principles and Methods," Data & Knowledge Engineering, vol. 25, no. 1-2, pp. 161-197, 1998.

[3] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP), 2019.

[4] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825-2830, 2011.

[5] ACM/IEEE, "Computing Curricula 2020: Paradigms for Global Computing Education," ACM/IEEE, 2020.

[6] European Commission, "The European Qualifications Framework for Lifelong Learning (EQF)," Publications Office of the European Union, Luxembourg, 2018.

[7] UNESCO Institute for Statistics, "International Standard Classification of Education: Fields of Education and Training 2013 (ISCED-F 2013)," Montreal, 2014.

[8] FastAPI, "FastAPI - Modern, Fast Web Framework for Building APIs," 2024. [Online]. Available: https://fastapi.tiangolo.com

[9] Groq, Inc., "Groq Cloud API Documentation," 2024. [Online]. Available: https://console.groq.com

[10] Hugging Face, "Sentence Transformers Documentation," 2024. [Online]. Available: https://www.sbert.net